



# IC-Cache: Efficient Large Language Model Serving via In-context Caching

# Background: The LLM Serving Challenge

## LLM Serving Today

### Huge resource demand

DeepSeek-R1: 671B parameters, 16 A100 GPUs

Qwen 2.5: 7B, Single GPU

### Quality vs. Cost tension

10× per-token cost: GPT-4o vs GPT-3.5

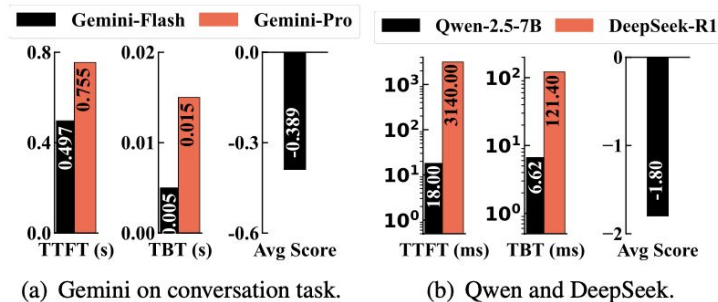
3× higher TBT, 65% Win Rate: Gemini 1.5 Pro vs Gemini 1.5 Flash

### Bursty workloads

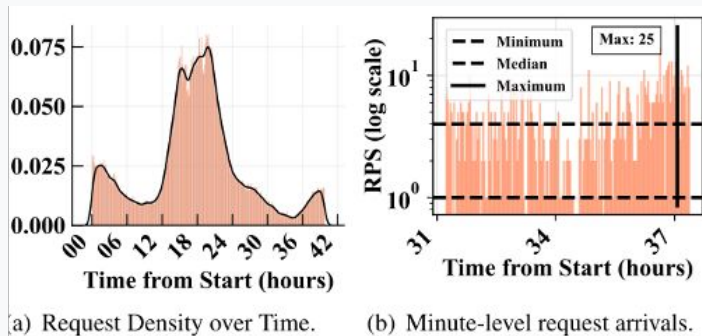
Peak loads 25× higher than off-peak in minutes

### High latency

Users face latency spikes even on optimized platforms

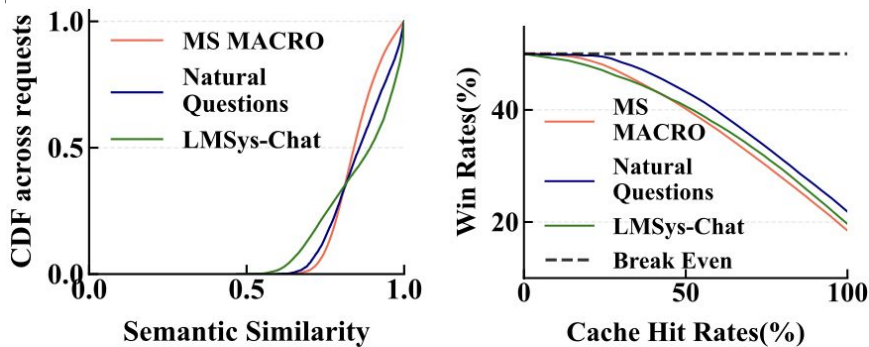


Quality-Efficiency Trade-off: larger models win on quality but cost 3× more in TBT latency



# Key Observation

Paper: Figure 3



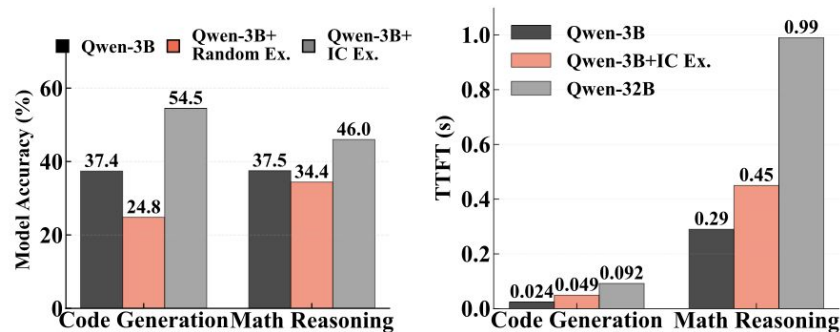
(a) Query similarity of three real-world datasets.

(b) Naive semantic caching based on request similarities hurts quality.



Fig 3: >70% of requests have cosine similarity >0.8 to another request — but naïve semantic caching drops win rate from 50% → 18%. Reusing responses directly causes off-topic replies.

Paper: Figure 4



(a) Response quality with examples.

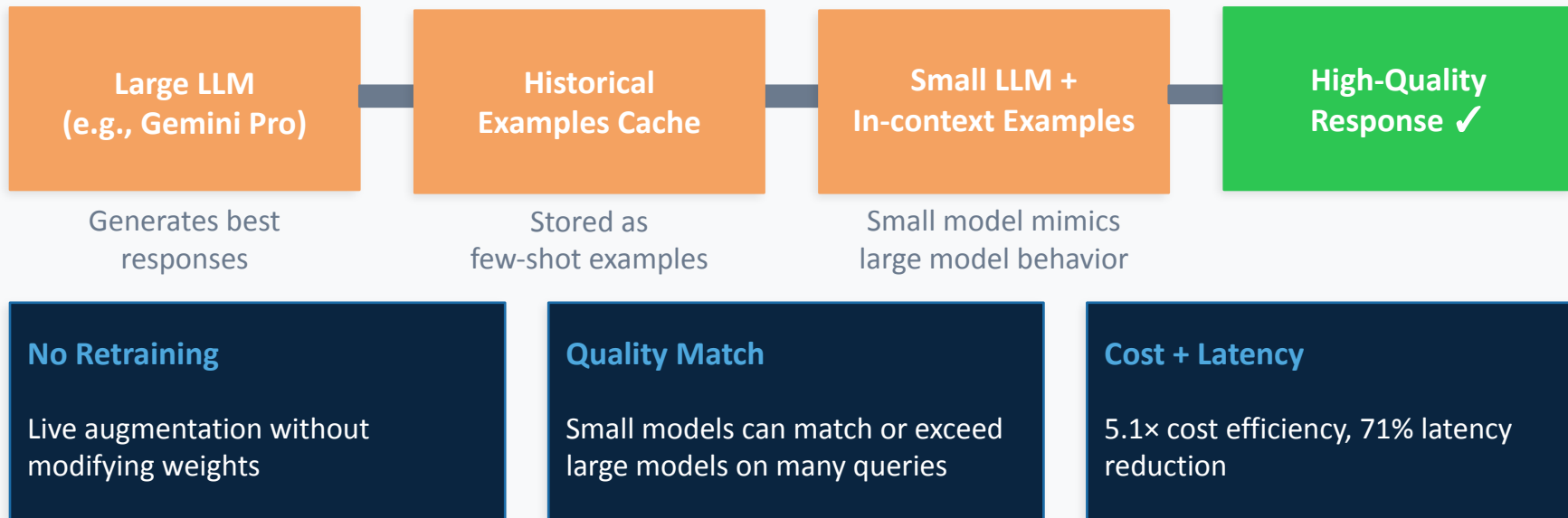
(b) TTFT performance.

✓ Fig 4: Using well-selected in-context (IC) examples from large models boosts Qwen-3B accuracy by 17pt on code gen & 12pt on math — while adding random examples hurts.

**Insight: Don't reuse responses — repurpose them as in-context examples to guide smaller models in real time.**

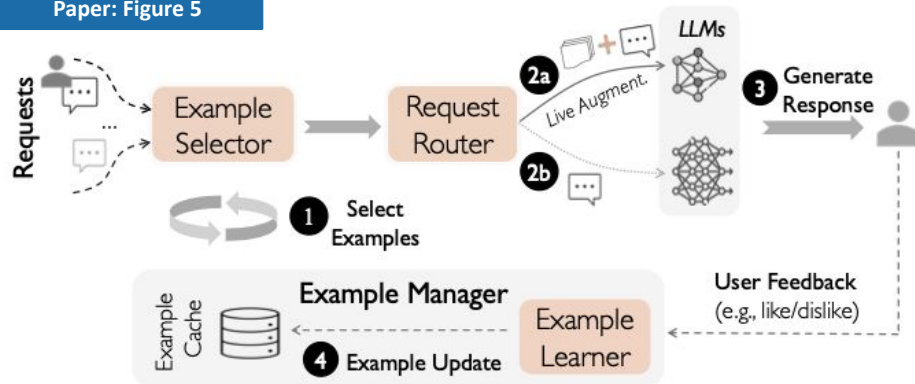
# The Core Insight

*Prepend historical request–response pairs from large LLMs as in-context examples to guide smaller LLMs — enabling live capability augmentation without any retraining.*



# IC-Cache Overview

Paper: Figure 5



**Figure 5. IC-Cache overview and request execution flow.**

## ① Example Selector

Finds high-utility historical request-response pairs from cache

## ② Request Router

Decides: small LLM + examples, or large LLM — adapts to load

## ③ Example Manager

Curates cache: replay, eviction, privacy controls

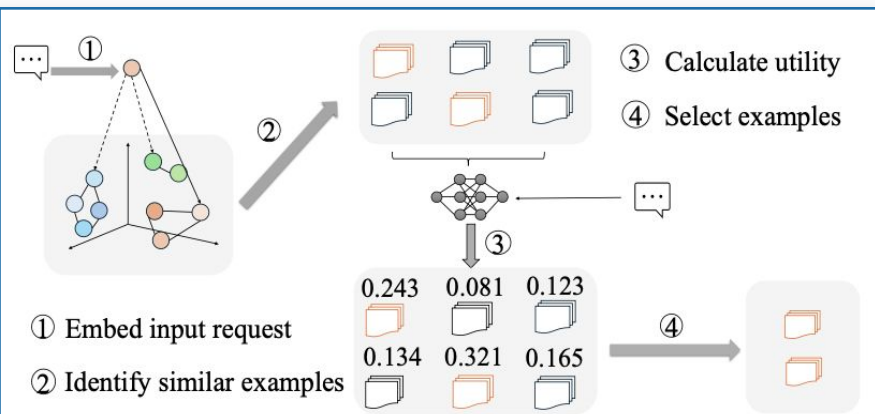
- **Retrieve**: pull high-utility historical request–response pairs from the example cache
- **Augment + route**: prepend the examples, then send the request to the most suitable model
- **Update**: cache new pairs, refine valuable examples offline, and evict stale ones over time

# Technical Deep Dive: Example Selector

Why relevance alone fails, and how to fix it

## Problem: Relevance $\neq$ Helpfulness

Pearson correlation between embedding similarity and actual helpfulness is only 0.04–0.22 across datasets. Relevance is a weak proxy.



**Figure 8. Overview of two-stage example selection.**

## Solution: Two-Stage Hierarchical Selection

### Stage 1: fast relevance filtering

Relevance filter via dense T5 embeddings + K-Means clustering ( $K=\sqrt{N}$ ). Narrows to small candidate pool efficiently.

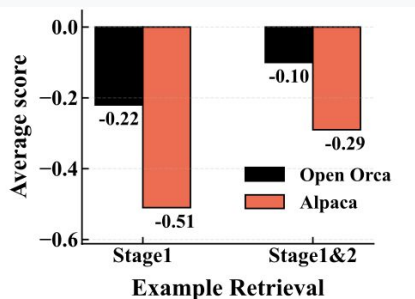
### Stage 2: lightweight proxy model for end-to-end helpfulness prediction

TinyBERT proxy model (<0.2% size of Qwen-7B) predicts per-request helpfulness score. Updated asynchronously from 1% sampled feedback.

# Technical Deep Dive: Example Selector

## Example Combination Selection — Adaptive Number of Examples

- Trade-off:** More examples → better quality, but longer input → higher TTFT + context overflow risk
- Dynamic threshold:** IC-Cache periodically samples requests to find optimal utility threshold (e.g., utility > 0.5) that maximizes efficiency
- Adaptive policy:** Number of examples adapts per query, balancing quality gain vs. latency overhead



**Figure 9.** *Two-stage example selection improves response quality.*

**Result: Two-stage selection improves average response score by 0.19–0.41 points vs. relevance-only, with <1% additional latency overhead.**

# Technical Deep Dive: Bandit-Based Request Router

## Why Not a Static Classifier?

- Cannot respond to real-time load changes
- Cannot adapt to model upgrades without re-labeling
- Requires expensive labeled data across all model choices

## Multi-Armed Bandit (MAB) Design

- Context:** Request + selected examples
- Arms:** Candidate LLMs (small+examples vs. large)
- Reward:** Response quality from user feedback
- Size:** Only 0.5M parameters — extremely lightweight

## Load-Aware Routing Strategy

### Normal Load (EMA < threshold)

Prioritize response quality. Many requests offloaded to small+IC model. IC-augmented small models can match large models on many queries.

### Overload (EMA > threshold)

Apply tanh-based corrective bias to logits:  $S_i(L) = \mu_i - \lambda_0 \cdot \tanh(\gamma L) \cdot C_i$ . Smoothly shifts routing to cheaper models. Also signals auto-scaling.

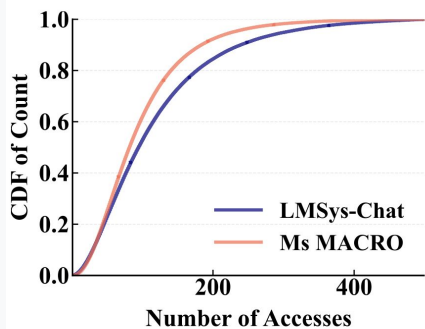
## Convergence Guarantee (Theorem1):

$P(\text{wrong model after } T \text{ rounds}) \leq (N-1) \cdot T^{-C}$  — the probability of selecting a suboptimal model decays polynomially with the number of requests.

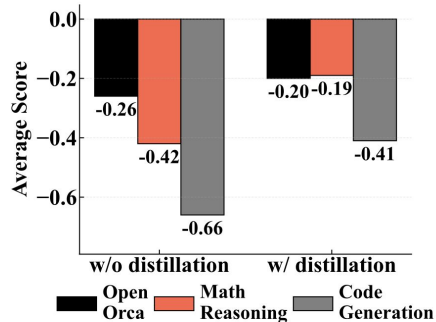


# Technical Deep Dive: Example Manager

Paper: Figures 10 & 11



**Figure 10.** *Example access exhibits long-tail distribution.*



**Figure 11.** *Example distillation improves final response quality.*

Fig 10 (left): Example access follows a long-tail distribution — a few examples reused hundreds of times, most rarely. This motivates selective replay rather than blanket regeneration.

Fig 11 (right): Cost-aware replay (distillation) improves response quality across Open Orca, Math Reasoning, and Code Generation — especially when examples still route to expensive models.

## Replay Gain Formula

$$G(e) = (1 - \text{quality}) \times \text{model\_cost}$$

replay when expected savings exceed replay cost.

# Example Manager (2/2) — Cache Size & Privacy

## Cache Size Management

### Knapsack Eviction

|           |                                |
|-----------|--------------------------------|
| Weight    | Plaintext size of the example  |
| Value     | # offloadings it enables       |
| Objective | Max value within memory budget |

Staleness decay:  $\text{EMA} \times 0.9$  / hour

**< 20 MB**

plaintext cache → near-peak performance

## Privacy

1

### API access control

Opt-in per request · domain-scoped sharing

2

### Client-side PII removal

spaCy NER · runs before caching · no raw data sent

3

### DP synthetic cache

Real pool → DP-synthesised pool for strictest privacy

§6.4:

*DP option tested — quality drops slightly, still beats baseline.*

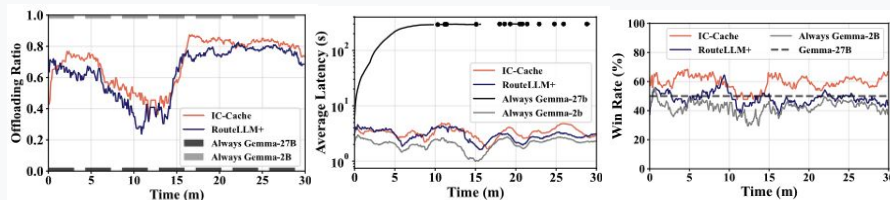
# Evaluation: End-to-End Performance

Millions of realistic requests across diverse model families

Setup: 16× NVIDIA A100 GPUs | Follows Microsoft real-world LLM serving trace | Gemini-1.5-Pro/Flash, DeepSeek-R1, Qwen2.5, Gemma-2, Phi-3 model families

| Task           | Dataset                | Example Size | Request Size |
|----------------|------------------------|--------------|--------------|
| Conversation   | Alpaca[66]             | 32,392       | 1,800        |
|                | lmsys-chat-1m [79]     | 273,043      | 15,170       |
|                | OpenOrca [49]          | 774,285      | 43,016       |
| Question Q&A   | MS MARCO [54]          | 808,731      | 101,092      |
|                | Natural Questions [41] | 300,000      | 7,830        |
| Translation    | WMT-16-PM [22]         | 600,000      | 1000         |
| Code Gen.      | NI2bash [75]           | 8090         | 609          |
| Math reasoning | Math500-Level5[45]     | 7500         | 5000         |

Table 1. Our evaluation data spans millions of realistic requests.



(a) Load Offloading (MS MARCO). (b) Serving latency (MS MARCO). (c) Response Quality (MS MARCO).

1.4–5.9×

Throughput  
Improvement

28–71%

Latency  
Reduction

9%

Higher Quality vs.  
RouteLLM

>50%

Win Rate vs.  
Larger Models

# Ablation Studies & Supplemental Results

## Impact of Example Cache Size (5%-100%)

IC-Cache reaches near-peak performance with just 12,056 examples for translation and 2,022 for code generation — under 20MB in plaintext.

## IC-Cache vs. RAG (LongRAG)

IC-Cache outperforms LongRAG. Combined IC-Cache + RAG achieves 62.4% win rate vs. 52.63% for RAG alone (Gemma-2-2B vs. 27B on MS MARCO).

## IC-Cache vs. Supervised Fine-Tuning (SFT)

SFT on in-domain data degrades out-of-domain performance (45.58% → 32.33% on out-of-domain Alpaca). IC-Cache achieves 47.25% in-domain and 46.69% out-of-domain without touching model weights — preserving general capability.

## Differentially Private Synthesis

With DP-synthesized examples (for strict privacy), IC-Cache still achieves 39–52% win rates vs. non-DP 40.5–57.3%. Marginal quality cost, significant privacy guarantee — practical for sensitive deployments.

# Related Work & Key Differentiators

| Approach                                   | Core Mechanism                                    | IC-Cache Difference                                                                                            |
|--------------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Semantic Caching<br>(Databricks, GPTCache) | Return cached response if similar query found     | IC-Cache uses examples to guide, not replace generation                                                        |
| Prefix KV Caching<br>(Gemini, DeepSeek)    | Reuse GPU KV cache for shared input prefixes      | Orthogonal & complementary; IC-Cache operates at request-level not GPU-level                                   |
| RAG (LongRAG)                              | Retrieve external documents for factual grounding | IC-Cache retrieves LLM responses, not documents — transfers compositional reasoning                            |
| RouteLLM                                   | Binary classifier: route to small vs. large model | IC-Cache augments small model capability + adapts routing online to real-time load — not just query difficulty |
| SFT / Knowledge Distillation               | Fine-tune small model on large model outputs      | IC-Cache is training-free, adapts to new domains in real-time without weight updates                           |

# Critical Analysis: Strengths & Weaknesses

## ✓ Strengths

### ✓ Novel problem framing

Repurposing historical requests for live capability augmentation is creative and impactful in terms of performance

### ✓ Complementary

Supports 3 LLM serving frameworks including HuggingFace Runtime, vLLM, and LangChain

### ✓ Minimal footprint

~20MB cache for saturated performance; <1% latency overhead for example selection

## ✗ Weaknesses / Limitations

### ✗ Cold-start problem

Needs a populated cache to work well; early deployment performance unclear

### ✗ Privacy-utility tension

DP examples reduce quality; non-DP caching raises data governance concerns at scale

### ✗ Context window limits

Prepending examples increases input length; problematic for very long base queries

# Future Research Directions

01

## Multimodal Models

Extending IC-Cache to vision-language models. Small VLMs often struggle with complex visual reasoning.

02

## Automated Example Quality Improvement

Beyond random replay, use RLHF or automated evaluation to actively generate better examples.

03

## Cold-Start Strategies

Ways to populate the cache efficiently before traffic accumulates.

04

## Theoretical Foundations of ICL Augmentation

What are the theoretical limits of in-context learning for capability transfer? Under what conditions can small models fully match large models with examples alone?

NSDI '24 · PAPER PRESENTATION

# NIRVANA

*Approximate Caching for Efficiently Serving  
Text-to-Image Diffusion Models*

---

Agarwal · Mitra · Chakraborty · Karanam · Mukherjee · Saini

Adobe Research · UIUC



# Text-to-image is exploding — and expensive

**2 B+**

Images generated by  
Adobe Firefly in 2023

**50**

Iterative denoising steps  
per image (modern DMs)

**~10 S**

Latency for one image  
on an A10g GPU

**10 ×**

Cost of A100 vs V100  
on AWS per hour

## The dilemma

**Smaller models → faster but lower quality. Bigger GPUs → faster but 10× more expensive.**

*Diffusion models are stuck on a quality–latency–cost Pareto frontier. We need a way OFF the frontier.*

# How diffusion models generate an image

## Iterative denoising

Start from pure **Gaussian noise** (state  $x_N$ ).

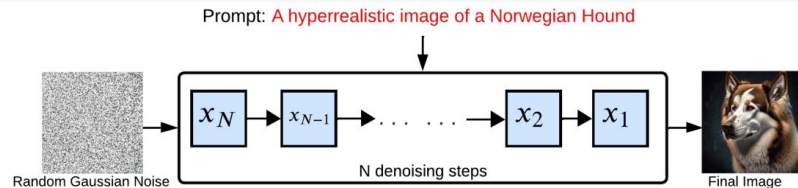
At each step, the U-Net predicts the noise and removes a little of it, conditioned on the text prompt via cross-attention.

After  $N \approx 50$  steps, you get the final image (state  $x_0$ ).

## Key properties

- Every step uses the same model and the same compute.
- The whole thing is sequential — you can't parallelize across steps.
- Latent diffusion: each  $x_i$  is a small tensor ( $\sim 144$  KB), not a full image.

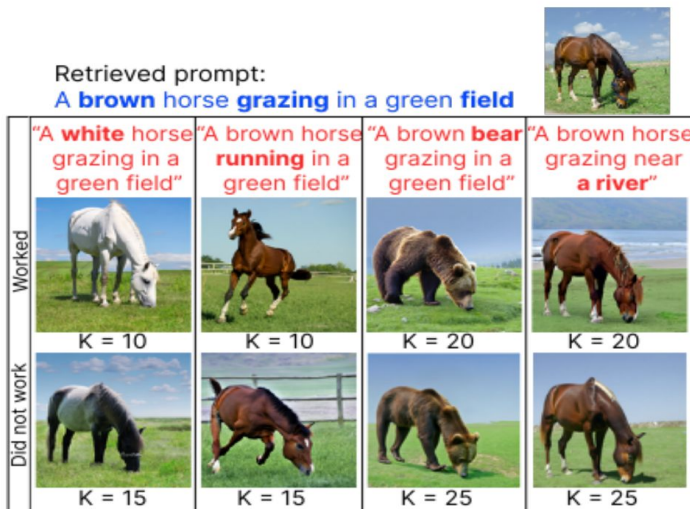
### Vanilla diffusion process



### THE OPPORTUNITY

**Every intermediate state  $x_i$  is just a tensor.**  
**We can save it. We can hand it to a different prompt.**

# Concepts in an image freeze early — and in a fixed order



## THE TRADE-OFF

### How many steps can we skip (K)?

**Skip K = 5** → works for almost any prompt, tiny savings

**Skip K = 25** → huge 50% savings, but only for VERY similar prompts

**Skip K > 25** → image is already locked in, can't recondition

# Real prompts repeat — a lot

*The authors analyzed 2 million real prompts from DiffusionDB to ask: is approximate caching even feasible in practice?*

## POPULARITY

### loosely packed cluster

#### Popular prompts persist

Popular prompt clusters stay popular for many days. 30–60% of top clusters survive a week.

## USER BEHAVIOR

### intra-session

#### Users repeat themselves

Within a 1-hour session, users issue near-identical prompts (a tight cluster of similarities at 1.0). Iterative refinement is the norm.

## THE LONG TAIL

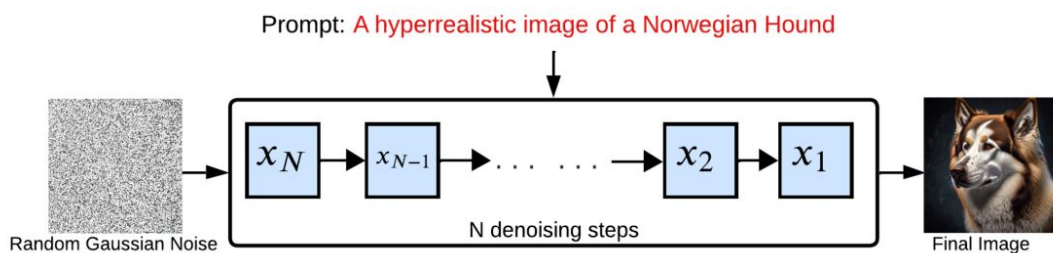
### across users

#### Strangers think alike

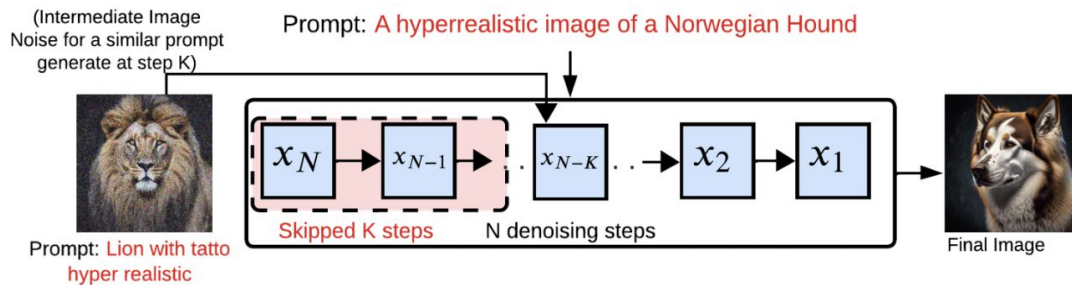
Even prompts from completely DIFFERENT users show high similarity — there is collective taste. Caching benefits the masses.

# Approximate caching, in one picture

## VANILLA DIFFUSION



## NIRVANA WITH APPROXIMATE CACHING



**Approximate is NOT retrieval. The cached state is still RECONDITIONED on the new prompt for the remaining  $50-K$  steps.**

# Picking K is a constrained optimization problem

## THE OBJECTIVE

### Minimize end-to-end latency L

Effective compute saving:

$$f_C = h(K) \cdot \frac{K}{N}$$

$h(K)$  = probability we can find a cached state at step K that we can recondition into a faithful image

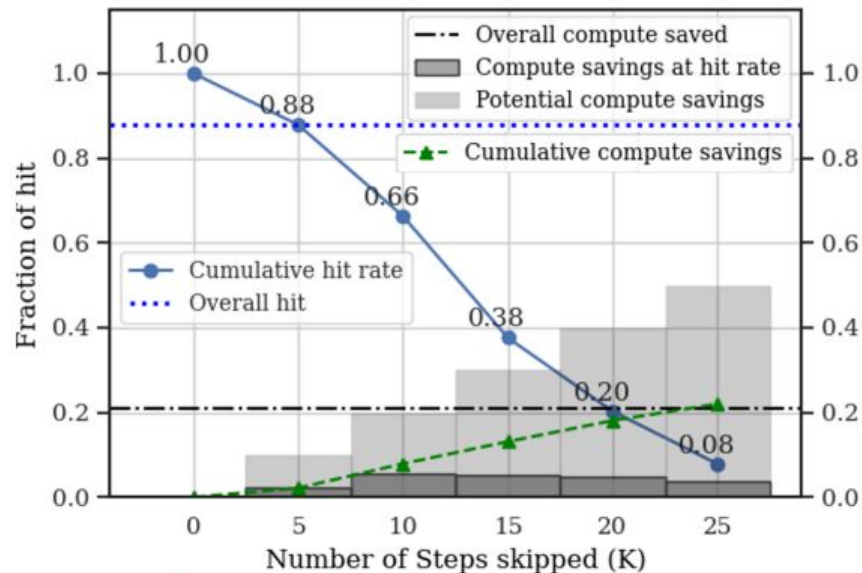
$K$  = number of denoising steps we get to skip

$N$  = total denoising steps ( $\approx 50$ )

Subject to:

$$\mathbb{Q}(I_K^c | \mathcal{P}_K, \mathcal{P}_Q) > \alpha \cdot \mathbb{Q}(I_0 | \mathcal{P}_Q) \quad \alpha = 0.9$$

Quality must stay within 90% of running vanilla diffusion from scratch.

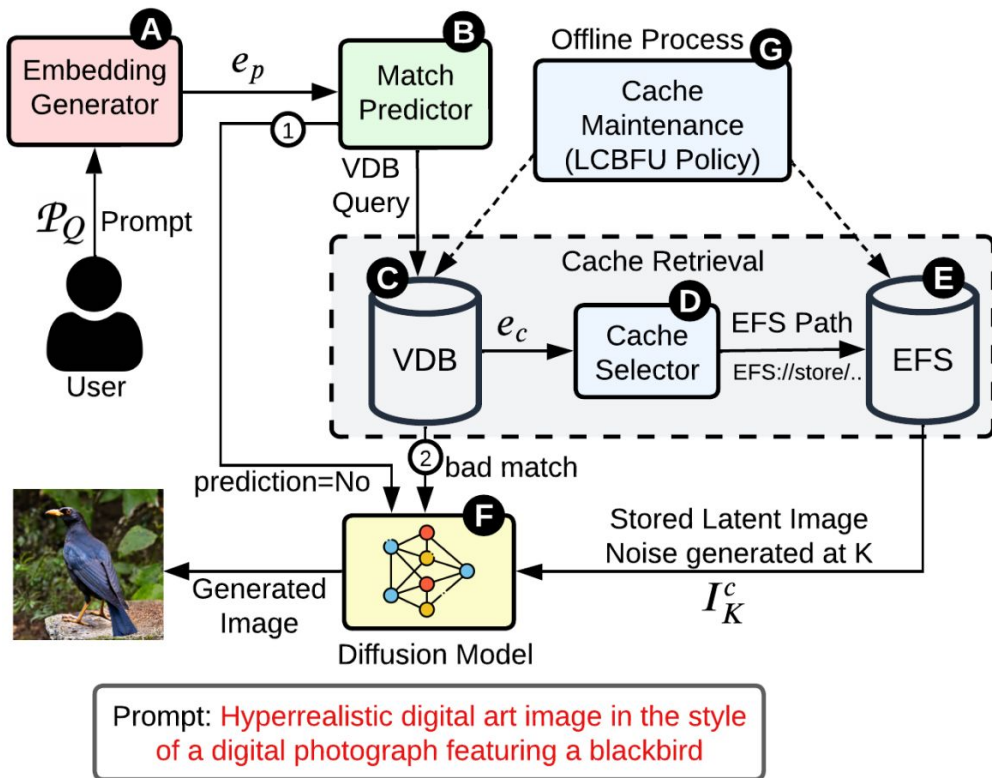


**Figure 15: Hit rate and compute saved across K's**

# NIRVANA architecture

## WALK-THROUGH

1. Embed the prompt with CLIP.
2. Match-predictor decides: is a useful hit likely?
3. Yes  $\rightarrow$  query vector DB for nearest neighbor.
4. Cache-selector picks K based on similarity score.
5. Fetch  $x_K$  from EFS, hand to diffusion model, run remaining  $N - K$  steps.



# Match predictor — avoiding wasted lookups

## The problem

On a cache miss, the vector DB lookup costs ~100 ms — pure waste. NIRVANA wants to avoid that.

## The solution

- A lightweight one-class SVM running on the CPU.
- Trained on all prompt embeddings currently in the cache (positive class only).
- Predicts: 'is this prompt close to anything we have?'
- SGD-based → retrains in 0.04 s for 10k records.

IN PRODUCTION

0.96

Precision

0.95

Recall

## Why not a Bloom filter?

Bloom filters answer: *'is this exact key here?'*

NIRVANA needs: *'is anything CLOSE to this here?'*

Approximate-nearest-neighbor doesn't fit a hash-based filter.



# Cache selector — picking K from similarity

## Offline profiling

For each candidate  $K \in \{5, 10, 15, 20, 25\}$ :

- Generate test images with cached states from the K-th step

- Find the MIN similarity score  $s_{\min}(K)$  at which all images still pass quality threshold  $\alpha = 0.9$

Result: a lookup table  $\text{sim} \rightarrow K$ , used at runtime

## Why these specific Ks?

**$K > 25$ :** image is mostly baked, can't recondition

**Finer-grained K:** more storage, more complexity, no proportional gain

*Each cached state is just 144 KB*

### RUNTIME LOOKUP

```
def cache_selector(s):
```

```
    if s > 0.95:          k = 25
```

```
    if s > 0.90:          k = 20
```

```
    if s > 0.85:          k = 15
```

```
    if s > 0.75:          k = 10
```

```
    if s > 0.65:          k = 5
```

```
    if else              :    k = 0
```

→ *higher similarity = skip more steps*

→ *similarity < 0.65: fall back to vanilla*

# LCBFU — a cache eviction policy made for diffusion

## Why traditional policies (LRU, LFU, FIFO) don't fit

*They treat all cached items the same. But in NIRVANA, a hit at  $K = 25$  saves  $5\times$  more compute than a hit at  $K = 5$ . Equal frequency does not mean equal value.*

### THE LCBFU SCORE

**Least Computationally Beneficial  
and Frequently Used**

$$\text{score}(i) = f_i \times K_i$$

$f_i$  = access frequency for item  $i$

$K_i$  = which step the cached state is from

*Evict the lowest-score items first.*

### WORKED EXAMPLE

Item A

$K = 25$ , accessed  $100\times$

**score = 2500**

Item B

$K = 5$ , accessed  $200\times$

**score = 1000**

***Evict B first — LRU/LFU would have kept it.***

# Image quality is preserved

## DiffusionDB benchmark

| Method            | FID ↓          | CLIP ↑       | Pick ↑       |
|-------------------|----------------|--------------|--------------|
| GPT-Cache         | 7.98           | 25.84        | 19.04        |
| Pinecone          | 10.92          | 24.83        | 18.92        |
| CRS               | 8.43           | 24.05        | 18.84        |
| SmallModel        | 11.14          | 25.64        | 18.65        |
| NIRVANA -w/oMP    | 4.94           | 28.65        | 20.35        |
| <b>NIRVANA</b>    | <b>4.68</b>    | <b>28.81</b> | <b>20.41</b> |
| <i>Vanilla DM</i> | <i>6.1–6.9</i> | <i>30.28</i> | <i>20.86</i> |

*Lower FID is better →*

*NIRVANA actually beats vanilla's own seed-to-seed variance.*

## Human user study (60 users · 1000 prompts)

### GPT-Cache



### NIRVANA



### Vanilla DM



*"Does the image faithfully represent the prompt?"*

# ...while cutting compute, latency, and cost

## 21%

**GPU compute saved**

*across DiffusionDB & SYSTEM-X workloads*

## 19.8%

**End-to-end latency reduction**

*and far lower tail-latency variance*

## 19%

**Dollar cost savings**

*amortized: GPU + EFS + vector DB*

### Throughput

1.27× more images per minute than vanilla — NIRVANA's hit/miss mix smooths out variability.

*Hit-rate ≈ 88% on production workloads*

### Latency stability

99th-percentile ÷ median latency:

GPT-Cache: **29.75×** (terrible tails)

NIRVANA: **1.21×** (very tight)

# How NIRVANA differs from prior approaches

## Retrieval caches

*GPT-Cache, Pinecone, CRS*

Just return the previous image for the closest prompt. Fast but quality collapses for any prompt drift.

**vs NIRVANA:** NIRVANA still RUNS the model — retrieval is just a head start.

## Smaller / faster DMs

*Distillation, Quantization*

Shrink the model itself: prune, quantize, distill, low-rank adapt.

**vs NIRVANA:** Orthogonal — NIRVANA stacks on top of any model.

## Faster samplers

*DDIM, PNDM, Parallel sampling*

Reduce N from ~1000 down to ~50 by smarter ODE solvers.

**vs NIRVANA:** Orthogonal — NIRVANA reduces from N down to N - K on top.

## DNN inference caches

*Clipper, Freezing-inference (Kumar et al.)*

Cache layer activations to avoid recomputing fixed layers.

**vs NIRVANA:** NIRVANA caches across PROMPTS, not layers; novel for DMs.

**Key positioning:** NIRVANA exploits a property unique to iterative generative models — savable, reusable intermediate states.

# Strengths and weaknesses

## STRENGTHS

**Conceptually novel.** Exploits a property unique to diffusion models that nobody had used before.

**Real production validation.** Evaluated on 8 weeks of Adobe's actual workload — not a synthetic trace.

**Compose-able.** Stacks on top of distillation, quantization, faster samplers, etc.

**Stable tail latency.**  $1.21\times$  p99/median vs  $30\times$  for retrieval baselines.

**Novel cache policy.** LCBFU is the first eviction policy where items have heterogeneous value per hit.

**Strong quality.** FID lower than vanilla's own seed-to-seed variance.

## WEAKNESSES

**Modest savings.** 21% / 19% is meaningful at scale but not transformative. The  $K=25$  bucket is rarely hit (8%).

**CLIP similarity becomes unreliable.** Long prompts can look similar in CLIP space yet differ semantically.

**K-buckets are hand-tuned.** Five fixed  $K$ s and an if-elif table — no learned or adaptive policy.

**Diversity collapse.** Reusing cached states risks homogenizing outputs.

**Architecture-specific.** Tied to U-Net latent diffusion; transformer-based DiTs may need rework.

# Where to go from here

## 1. Learn the K selector

Replace the hand-tuned if-elif table with a small predictor that takes (prompt, candidate, similarity) → optimal K. Could likely close the gap to ideal savings.

## 2. Better embeddings for long prompts

CLIP fails on long prompts. Use a fine-tuned text encoder, decompose prompts into sub-concepts, or use multi-vector retrieval.

## 3. Compositional caching

Cache states keyed by individual concepts (subject, style, layout) rather than full prompts, then mix retrieved states from different cache entries.

## 4. Extend to DiTs and video diffusion

Test whether the 'concepts freeze early' property holds for transformer diffusion models and for video DMs, where compute costs are even more extreme.

## 5. Diversity-aware eviction

Augment LCBFU with a coverage term so the cache spans the prompt space rather than collapsing on popular themes.

## 6. Cross-tenant federated cache

Adobe already showed cross-user caching helps. A federated cache across tenants (with privacy guarantees) could amplify savings dramatically.

***Approximate caching turns prompt similarity into compute savings — without sacrificing quality. Questions?***